

DBARS - MASTER REVISION SHEET

Demand Based Bus Allocation & Routing System | Everything you need the night before

1. Project summary

DBARS turns BMTC's published bus timetable into three products: a commuter journey planner with transfers and e-ticketing, an operations planner that computes the minimum fleet and crew a timetable needs, and a GTFS open-data feed any journey planner can consume. FastAPI + MongoDB backend, React + TypeScript frontend, Docker, 161 tests in CI.

Figure	Value
Dataset	6,737 route-directions 3,940 buses 4,883 stops
Search index	655,438 ordered stop pairs over 3,946,744 route segments
Search latency	median 0.94 s p90 1.52 s
Fleet result	6,849 buses across 44 depots
Crew result	21,453 pieces -> 15,636 duties ratio 2.28 37.2% split
GTFS feed	3,713 routes 47,295 trips 1,459,066 stop_times ~14.4 MB ~11 s
Code	13,562 lines backend 8,459 frontend 2,440 tests 82 endpoints 161 tests

2. Architecture

```

Browser (React SPA)
  | JSON over HTTP, Bearer token
API layer          app/api/, app/auth/    HTTP only, no business rules
  |
Service layer      app/services/          business rules; ONLY layer touching MongoDB
  |
Data layer         app/db/                \
MongoDB (17 collections)                  Engine layer  app/ml, app/gtfs, app/tracking
                                           NO database - reads dataset/*.csv directly
  
```

THE KEY DECISION: engines read files, not the database.
 train() runs BEFORE init_db(), and init_db() cannot raise.
 Result: route search, blocking, crew and GTFS all survive a database outage.

3. Technology stack

Layer	Technology
Backend	Python 3.12, FastAPI 0.115, Uvicorn, Pydantic 2.10, Motor 3.6
Frontend	React 19, TypeScript 5.7, Vite 6, Tailwind 3.4, Leaflet 1.9, Recharts 3
Database	MongoDB - 17 collections, no ORM, accessed directly through Motor
Open data	GTFS static + GTFS-Realtime (gtfs-realtime-bindings, protobuf)

Infra	Docker Compose (backend/frontend/mongo), Nginx, Render, Vercel, GitHub Actions
Mobile	Capacitor 8 (Android shell around the built web bundle)
NOT used	No ORM. No JWT library (hand-written). No ML framework in serving. No LLM. No payment gateway. No Redis, queue or cache server.

4. Complete data flow

User action

```
-> React handler -> apiFetch (lib/apiClient.ts)
-> CORSMiddleware -> RateLimitMiddleware (120/min per IP)
-> FastAPI router -> Pydantic validation (422 if bad)
-> Depends(get_current_user) / require_role(...) [protected only]
-> endpoint (app/api) -> service (app/services) or engine (app/ml)
-> MongoDB or dataset/*.csv
-> JSON response -> setState -> re-render -> UI
```

JOURNEY SEARCH specifically:

```
resolve both stop names (alias + fuzzy, with confidence)
-> stop_pair_to_segments([A,B]) direct hit?
-> else build one-transfer chains via a shared interchange stop
-> reject transfers > 1.2 km apart (_interchange_is_walkable)
-> re-anchor the whole journey as ONE sequence (_reanchor_legs)
-> score, rank, deduplicate -> best_match + alternatives
```

5. Ten most important files

#	File	Why it matters
1	backend/app/main.py	Entry point: create_app(), lifespan(), startup order
2	backend/app/ml/predictor.py	The search engine - indexes, ranking, transfers (1,600+ lines)
3	backend/app/ml/blocking.py	Fleet size and dead kilometres
4	backend/app/ml/crew.py	Crew duties cut from vehicle blocks
5	backend/app/ml/route_geometry.py	Anchored stop resolution - shared by three subsystems
6	backend/app/gtfs/builder.py	Streams the 1.46M-row GTFS feed into a zip
7	backend/app/tracking/ingest.py	Vehicle feed loop + honest source reporting
8	backend/app/auth/auth.py	Hashing, tokens, three signing domains, roles
9	backend/app/services/waybill_service.py	The money path: pricing, idempotent sync, reconciliation
10	backend/app/core/shakti.py	The single shared free-travel eligibility rule

6. Ten most important functions and classes

#	Name	What it does
1	BMTCBusPredictor.predict()	The product: two stop names in, best bus out
2	_interchange_is_walkable()	Rejects transfers whose two stops are >1.2 km apart

3	resolve_stop_sequence()	Two-pass anchored resolution; stops routes teleporting
4	chain_blocks()	Greedy trip-to-bus chaining; provably optimal without an idle cap
5	concurrency_lower_bound()	Peak simultaneous trips = hard floor on fleet size
6	cut_block_into_pieces()	Dynamic program: fewest legal crew pieces per block
7	combine_pieces_into_duties()	Bounded greedy pairing into split duties (36 s -> 1.6 s)
8	is_shakti_eligible()	One rule, used by both the app and the conductor
9	issue_tickets()	Idempotent offline sale recording with per-item results
10	estimate_arrival()	Route-following ETA with a derived confidence label

7. Database summary

Aspect	Detail
Type	MongoDB document database, accessed via Motor (async). No ORM.
Optional?	YES. <code>init_db()</code> logs and continues if unreachable; engines never use it.
Collections	17. Key ones: users, votes, tickets, waybills, conductor_tickets, travel_passes, audit_events, service_alerts, trip_shares
Critical index	conductor_tickets (waybill_id, client_ticket_uuid) UNIQUE -> this is what makes offline ticketing idempotent
TTL indexes	trip_shares expire after 6 h (physically deleted); usage_events after 90 days; audit_events deliberately have NO TTL
Relationships	By stored id only. No joins, no foreign keys - integrity is the application's responsibility
Transactions	NONE. A crash mid-sign-off could leave a waybill inconsistent

8. API summary

Group	Highlights
Public	POST /predict, GET /autocomplete, /routes, /metrics, /health, /gtfs/static.zip, /gtfs-rt/service-alerts.pb, /tracking/*
Auth	POST /auth/register, /auth/login, /auth/refresh; GET /auth/me
User	POST /votes, /api/tickets/purchase, /crowding/report, /safety/share-trip
Conductor	POST /conductor/sign-on, /tickets/sync, /sign-off, /verify-pass
Depot/Admin	GET /depot/blocking-plan, /depot/crew-plan, /depot/waybills, /admin/shakti/claim, /admin/revenue/reconciliation, /admin/audit
Machine	POST /avl/ingest (X-AVL-Key), POST /sms/webhook (Twilio signature)
Total	82 endpoints, 14 routers. Interactive docs at /docs
Known gap	POST /train is unauthenticated and expensive - the clearest hole

9. AI summary - read this twice

There is no AI in the ROUTING ENGINE - keep the two apart

The thing that predicts buses has no LLM, no neural network, no trained model file and no inference API. It is an inverted index plus an ordered stop-pair index with a hand-weighted scoring function - classical information retrieval and graph search - benchmarked against three baselines. A separate AI layer was added later (section 16). Conflating them is what invites the question you cannot defend.

The benchmark was single-label: it demanded the exact route each sample was drawn from, on pairs where a MEDIAN OF 22 buses are all correct. A perfect system picking uniformly among valid buses would have scored only 0.16. It was replaced with relevance-set scoring. All four models now run on an identical N=120.

Model	Coverage (valid-route)	Strict top-1	Note
TFIDFCosine	1.0000	0.1667	Baseline ranker
OrderedStopFuzzy	1.0000	0.1583	Baseline ranker
DistanceAwareRouteRanker	1.0000	0.1583	Baseline ranker
LiveTransferSearch (SHIPPED)	1.0000	0.1917	What the API actually returns

CRITICAL: the coverage column is NOT accuracy. The relevance set is read from stop_pair_to_segments - the SAME index the search uses to find its candidates - so "did it return a valid route?" is close to tautological and cannot drop below 1.0 without an outright bug. It proves the search never fails to find something valid; it says nothing about quality. The HEADLINE metric is rank_quality, because it is the only one the system can actually fail - and at baseline it did, at 0.57.

Rank-quality metric (HEADLINE)	Baseline	After Phase 3	Threshold
best_option_rate	0.57	0.7167	>= 0.70
mean_percentile_rank (0 = best)	0.14	0.0437	<= 0.10
within_2_stops_rate	0.90	0.9917	>= 0.92
ndcg_at_5	not recorded	0.9810	-
coverage (must not regress)	1.0000	1.0000	>= 0.95
p90 search latency	1.52 s	1.19 s	<= 1.6 s

Levers applied: stop span made the primary ranking signal, a $\log_{10}(\text{trip_count})$ frequency term, and a penalty for routes whose span is far above the minimum for the pair. Each was applied in isolation and kept only if the metric improved. Verified by scripts/verify_accuracy_plan.py, which re-derives the numbers live on a fixed seed rather than trusting metrics.json - 21 criteria, exit 0. Criteria 1.7 and 1.8 fail the build if the headline is ever set back to a coverage metric.

How to say this - NEVER "I improved accuracy from 20% to 100%"

That phrasing is indistinguishable from moving the goalposts, and an examiner will say so in one sentence. Say instead: "The benchmark was single-label - it demanded the exact route each sample was drawn from, on pairs where a median of 22 buses are all correct. I measured its ceiling at 0.16, so it could not discriminate. Replacing it showed the search always returns a valid route, but I report that as COVERAGE rather than accuracy, because the validity test reads the same index the search does - it is close to circular. The metric that could actually be failed was whether the BEST valid route ranks first: 57% at baseline, 72% after weighting stop span and frequency." Showing that you found the limits of your own metric is worth more than any number.

10. Security summary

Exists	Missing / weak
PBKDF2-SHA256 passwords, 16-byte random salt HMAC-SHA256 signed tokens Three separate signing key domains Token type checking (access/refresh/ticket/pass) Role-based access control on every write	POST /train unauthenticated <-- worst No token revocation (logout is client-only) Tokens in localStorage (XSS exposure) No login throttle or account lockout Password minimum only 6 characters

Role forced to commuter at registration Pydantic input validation everywhere Per-IP rate limiting (120/min) Twilio signature + AVL key verification Server-side pricing and totals Idempotent money writes (unique index) No secrets in source; .env gitignored	No email verification In-memory rate limiter (breaks with workers) No global exception handler No HTTPS enforcement in the app No CSRF protection (low risk - Bearer, not cookies)
---	--

11. Top 20 professor questions

1. What is your project?

A transit platform on BMTA's published timetable: journey planning for commuters, fleet and crew planning for depot managers, and a GTFS open-data export.

2. What problem does it solve?

Commuters cannot find routes among 3,940 buses and 4,883 stops; operators get nothing operational from a commuter app; external planners need standard data.

3. Explain the architecture.

Four layers: API (HTTP only), services (business rules, only layer touching MongoDB), engines (pure computation over CSV), data. Dependencies point one way.

4. Where does it start?

backend/app/main.py - create_app() builds it, lifespan() verifies files, trains the engine, warms caches, starts vehicle ingest, then connects to MongoDB.

5. Why train before connecting to the database?

So route search works before the database is even attempted. init_db() cannot raise, so an outage cannot abort startup.

6. How does the search work?

An ordered stop-pair index built at startup - 655,438 pairs. Direct routes are a dictionary lookup; transfers intersect the routes serving each end.

7. Why ordered pairs?

Direction is the whole question. A bus passing both stops in the wrong order is not an answer.

8. What is a block versus a duty?

A block is one bus's day - pull out, chain of trips, pull in. A duty is one person's day. A plan that saves buses by stretching them across 14 hours saves nothing if no roster can staff it.

9. How many buses and crew does the network need?

6,849 buses across 44 depots, and 15,636 daily crew duties - a ratio of 2.28, which matches real transport undertakings.

10. How do you know the fleet number is right?

I compute a hard lower bound - peak simultaneous trips - so the output states whether it is provably minimal rather than just the best found.

11. Where do your assumptions live?

In BlockingParameters and CrewParameters, echoed into every API response so nobody mistakes a modelled number for a measured one.

12. Why MongoDB?

Records evolved during development and a schema-free store absorbed that without migrations. The trade-off is real: no joins, no foreign keys, so integrity is my job.

13. What if the database is down?

Route search, blocking, crew and GTFS keep working - they read files. Anything needing stored data returns a 503 that says so, or an empty result with a note.

14. How does authentication work?

PBKDF2-hashed passwords; HMAC-SHA256 signed tokens; three separate signing keys so a leaked ticket key cannot mint a session; roles checked by a FastAPI dependency.

15. Where is the AI?

There is none. Some UI text says AI and that is wrong. It is classical information retrieval and graph search, benchmarked against three baselines.

16. How accurate is it?

100% lenient top-1 under relevance-set scoring (returns a valid bus in every test search); 71.7% best-option rate and 99.2% within 2 stops of optimal on rank quality; 19.2% on the old single-label exact match. Defend the measurement methodology: separating validity from ranking quality.

17. Is the bus tracking real?

No - simulated by default. Every observation carries an is_live flag, and the GTFS-Realtime vehicle endpoint returns 503 rather than publish simulated data as real.

18. What happens with 10,000 users?

It would not hold. It is single-process: the rate limiter, caches and vehicle store are in memory. Scaling needs that state moved to Redis first.

19. What is the weakest part?

Zero frontend tests across 8,459 lines, including the offline queue that handles money. The backend has 161 tests, so the asymmetry is indefensible.

20. What was the hardest problem?

A route map drawing 69 km lines for 19 km journeys - unanchored stop resolution matching the wrong Kodihalli. Fixing it exposed a worse bug: transfers matched by name alone, offering a changeover between two Avalahallis 28.6 km apart.

12. Top 20 recruiter questions

1. Tell me about this project.

A transit platform for Bengaluru's buses. It plans journeys for commuters, computes the minimum fleet and crew for depot managers, and publishes the data as GTFS. FastAPI, React, MongoDB, Docker, 161 tests in CI.

2. What was your role?

[State your own contribution accurately - what you designed, decided and debugged. Do not overstate; being able to explain WHY beats claiming authorship.]

3. What is the scale?

6,737 route-directions, 4,883 stops, a 655,000-entry search index, 82 API endpoints, ~22,000 lines of code.

4. What was the architecture?

Four layers with a one-way dependency rule. The key decision was making the computation engines pure functions of files, with no database dependency.

5. Why does that matter?

The core features survive a database outage, and all that logic is unit-testable with no infrastructure - which is why 161 tests run in CI with no MongoDB container.

6. What was the hardest technical problem?

A route map drawing straight lines across the city. Distances beside the map were correct, so only the drawing was wrong - which is why it hid for so long. Behind it was a worse bug: transfers matched purely on stop name.

7. How did you fix it?

Two-pass anchored resolution, then a geometric check rejecting changeovers over 1.2 km. I got the fix wrong twice first - I ran the check after re-anchoring, which made it always pass, then filtered a fixed shortlist and made a real journey unroutable.

8. Give me a performance example.

Crew pairing was quadratic over 21,453 pieces - 36 seconds. Binary search on the spreadover window plus a path-compressed skip list took it to 1.6 seconds, and it produced slightly fewer duties.

9. How do you handle unreliable networks?

Conductor sales are offline-first. Each sale carries a client-generated UUID and replays until acknowledged; a unique database index makes replays idempotent, and the response is per-item so the device clears only what was confirmed.

10. Why per-item responses?

Clearing the whole queue on a 200 is exactly how offline sales disappear, and a lost cash sale is indistinguishable afterwards from one that never happened.

11. How would you scale it?

Name the constraint first: single-process, with in-memory rate limiting and caches. Move that to Redis, then cache popular route queries, then split the read-only engines onto their own instances.

12. What is the first bottleneck?

The one-second CPU cost of a route search. It saturates the thread pool long before MongoDB becomes a problem.

13. How would you debug production?

/health and /tracking/source give me predictor state, feed status and last error. The honest gap is that I have logs and nothing else - no request ids, no metrics.

14. Why FastAPI?

Async by default, validation from type hints so malformed input never reaches business logic, and automatic OpenAPI docs.

15. Why no ORM?

The data is document-shaped and simple. An ORM would have added a layer without removing work.

16. How did you test it?

Unit tests on hand-built fixtures, plus dataset tests asserting plausibility ranges - because every wrong version of the fleet engine still ran cleanly and produced confident wrong numbers.

17. What would you do differently?

Write frontend tests from the start, add observability early, design the shared-state story before building caches, and name things accurately - calling index-building 'training' created confusion I had to correct.

18. What did you learn?

That the hard part was making the system honest, not making it work. Several bugs produced plausible, confident, wrong output without crashing.

19. What are you proudest of?

The operations engines. Computing minimum fleet and crew duties from a published timetable is work a transport corporation actually does, and no consumer app attempts it.

20. What is missing?

Frontend tests, token revocation, real payment, real vehicle tracking, and horizontal scalability. All are documented rather than hidden.

13. Biggest weaknesses

#	Weakness	Severity
1	Zero frontend tests - 8,459 lines, including the offline money queue	High
2	POST /train unauthenticated and triggers a full retrain	High
3	No observability - no global exception handler, request ids, metrics or tracing	High
4	Single-process only - in-memory rate limiter, caches and vehicle store	High
5	No token revocation; tokens in localStorage	Medium
6	No database transactions - a crash mid-sign-off leaves a waybill inconsistent	Medium
7	Misleading naming - train() trains nothing; the AI assistant parses strings	Medium
8	1.6 MB frontend bundle with no code splitting	Medium
9	Weak password policy, no email verification, no login throttle	Medium
10	predictor.py over 1,600 lines mixing four responsibilities	Low

14. Top improvements, ranked

1. Gate POST /train behind require_role(ADMIN) - one line, closes the clearest hole.
2. Add a global exception handler with request ids and structured logging.
3. Add frontend tests, starting with lib/offlineQueue.ts because it handles money.
4. Correct the misleading naming (train, AI assistant) so no false claim is invited.
5. Move rate limiting and caches to Redis - the precondition for multiple workers.
6. Route-level code splitting to cut first-load time.
7. Login throttling and a stronger password policy.
8. Give the driver role an interface, or remove it.

15. The three explanations

30 seconds

"DBARS is a full-stack transit platform for Bengaluru's bus network. It takes the published BMTC timetable and does three things: plans journeys for commuters including transfers, tells depot managers the minimum buses and crew that timetable needs, and publishes the whole dataset as GTFS for external journey planners. FastAPI, React, MongoDB, Docker, 161 tests in CI."

1 minute

"DBARS is built on BMTC's published timetable - about 6,700 route-directions across 4,900 stops. There are three products in it. For commuters, a journey planner that answers 'which bus from A to B' including one-transfer journeys, plus e-ticketing with QR codes and the Karnataka Shakti free travel scheme. For the operator, something no consumer app does: it computes the minimum fleet that can run the timetable, the kilometres run empty, and the crew duties needed to staff it - about 15,600 duties for 6,800 buses. And it exports everything as GTFS so any journey planner can use the data. Backend is FastAPI with MongoDB, frontend is React and TypeScript,

containerised with Docker, 161 backend tests in CI. The design decision I would highlight is that the computation engines read files rather than the database, so the core features keep working when MongoDB is down."

3 minutes

Give the 1-minute version, then add these five points:

- **Architecture.** Four layers, strict dependency direction. Engines are pure computation over CSV with no database dependency - which is why 161 tests run in CI with no MongoDB, and why startup trains the engine before attempting a database connection.
- **How search works.** Not a trained model - an inverted index plus 655,438 ordered stop pairs built at startup. Direct routes are a dictionary lookup; transfers intersect the routes serving each end. About a second of CPU, so it runs on a worker thread.
- **The hardest bug.** Route maps drew straight lines across the city - unanchored stop resolution matched the wrong Kodihalli 28 km away, drawing 69 km for a 19 km journey. Fixing it exposed transfers matched purely by name, offering a changeover between two Avalahallis 28.6 km apart.
- **Honesty by design.** Vehicle positions are simulated by default and carry an `is_live` flag; the GTFS-Realtime vehicle endpoint returns 503 rather than publish them, because the format cannot mark data as simulated and a consumer could not tell.
- **What I would fix.** No frontend tests, POST /train unauthenticated, no token revocation, and single-process only - scaling needs shared state moved to Redis.

16. The AI layer - and how to describe it honestly

Added after the routing engine and deliberately separate from it. Answers questions about the codebase, the documentation and live transit state.

Component	What it is
Retrieval	AST-chunked code index + documentation index; BM25 fused with vectors by Reciprocal Rank Fusion
Tools	8 read-only DBARS tools, role-gated at one chokepoint
Agents	codebase / operations / travel, dispatched by intent
LLM	Gemini when a key is set; a grounded deterministic model otherwise (the default)

The sentence that keeps you honest

Only 1.8% of benchmark answers actually invoke the language model. The orchestrator is an intent router into hand-written agents; the LLM is the fallback for queries no agent claims. Swapping the offline model for live Gemini moved mean answer correctness from 0.8150 to 0.8146. It is retrieval and templating with an LLM available - calling it LLM-powered would overstate it.

The 0% hallucination rate follows from that. Most answers are assembled from retrieved evidence, so there is structurally nothing to hallucinate with. It is a property of the architecture, not evidence that a generative model was tested and found truthful.

16.1 Security - the strongest story here

The layer shipped with NO authentication on any `/ai/*` endpoint and was a working bypass of `require_role()`. GET `/depot/blocking-plan` returned 401 to an anonymous caller while POST `/ai/chat` answered the same question with the depot fleet figures and a 200. Every `require_role` occurrence in `app/ai` was a string in a prompt or a regex, not an enforced check.

- Identity now comes from the verified token, never the request body.
- Tools are gated at `ToolRegistry.execute_tool()` - one chokepoint, fails closed.
- Session memory is keyed `user_id::session_id`, closing an IDOR that leaked journeys.
- Observability and evaluation endpoints are admin-only.
- 19 regression tests pin the 401-versus-200 pair.

16.2 Defects it exposed in the routing engine

Symptom	Root cause
dasarahalli metro station returned Vajarahalli - the far end of the line	Matching compared spelling, not place
Whitefield to Kengeri: 86 stops / 85.9 km instead of 33 stops / 41.4 km	Kengeri Bus Station counted as inexact despite being 0.22 km from Kengeri
78 stops / 146 km beat 85 stops / 62 km	Stop count used as a proxy for distance
Sapthagiri to Reva went via the centre	Search could not see past one interchange

29.8% of results scored the pick lower than an alternative shown below it	Ranking and confidence were unrelated formulas
First depot question took 10-15 seconds	Crew plan was never warmed at startup

430 tests pass. best_option_rate held at 0.7167 through every change and ndcg@5 improved from 0.9810 to 0.9820. Say that the fixes were verified NOT to move the headline metric - that is what makes them safe to claim.

16.3 What did not work - volunteer this

Semantic embeddings (all-MiniLM-L6-v2) were the obvious upgrade to a lexical index. Measured on identical content: 8/10 with query expansion either way, and 4/10 versus 5/10 without it - no improvement, for 80x the load time and 25x the memory. The model is trained on sentences, not source code. Reverted to the deterministic provider. Being able to say I tried it, measured it, and it did not help is worth more than any claim that it did.

The three questions most likely to catch you out

1. "Where is the AI?" - Two answers, kept apart: the ROUTING ENGINE has none, it is classical IR; a separate AI layer exists (section 16) and only 1.8% of its answers reach an LLM. 2. "How accurate is it?" - 100% validity (relevance-set), 71.7% rank quality, 19.2% exact match; defend the measurement design. 3. "Is the tracking real?" - No, simulated by default, and the code is built so it cannot be published as real.